Comparison Between Pseudocount and Soft-Imputation Left-censored Zero Handling

By: Christian Anderson
30 October 2026

Abstract. **Background:** Compositional data, such as gut metagenomic relative abundances, lie on a simplex and contain spurious correlations that analyses assuming Euclidean geometry do not account for, so they are usually transformed with the centered log-ratio (CLR). The CLR is undefined at zero, yet biological compositions are sparse (often more than 50% zeros), and these zeros are often due to technical effects such as limited sampling depth (left-censoring). They must therefore be replaced before the transformation. **Methods:** I compared replacing zeros with a small constant (a pseudocount) to imputing them with SVD-based soft imputation. I simulated Dirichlet-multinomial count data ($n = 100$ samples, $p = 5000$ features) with a group effect multiplied to a subset of features, tuned each method over a grid of its parameters, and compared sensitivity (power) at a false discovery rate (FDR) below 0.05 from per-feature t-tests on the CLR-transformed data. **Results:** XXX (best settings of each method, and their sensitivity and FDR). **Conclusions:** XXX (which method performs better, and under what conditions).

Keywords. Compositional Data, Zero Imputation, Centered Log-Ratio, Dirichlet-Multinomial

Introduction

Compositional data lie on the simplex: each value is between 0 and 1, and the values for each observation sum to 1. As a result, when one variable goes up, another goes down. The compositional constraint leads to spurious negative correlations, meaning a change in one variable causes a decrease in another, even if they are completely independent of each other. For example, if the first value of $[0.8, 0.1, 0.1]$ decreases, the remainder must increase, giving $[0.6, 0.2, 0.2]$; analyzing this data as it is would lead us to believe that the first feature is negatively correlated to the other two. One way to avoid them is to transform the simplex, also called Aitchison space, to Euclidean space, so it will work with statistical tests and tools, such as the t-test and PCA.

Most biological compositional data are also sparse, meaning most features are so rarely observed that the majority of their values are zero. This means the feature was either not there, or the technical process was not sensitive enough to record it. We assume the latter (aka, left-censoring) and treat these zeros like missing values To convert the simplex into Euclidean space, the most common method is the centered log-ratio (CLR) transformation, which centers the log abundances on their geometric mean[1]:

$$\mathrm{clr}(x_j) = \log(x_j) - \mathrm{mean}(\log(x))$$

However, $\log(0)$ is undefined, and because the data are sparse, the result would contain a large number of missing values.

Motivation

Many methods exist to deal with these zeros, two stand out for being simple and elegant. In the first method, a small value, a pseudocount, simply replaces the zeros. A value of $\frac{\min(x_{x\neq 0})}{2}$ is often used, but the choice may have large downstream effects. Second, the zeros can be imputed using soft imputation, which infers values the zeros can take such that the overall variance of the matrix is not altered. This method iteratively calculates the singular value decomposition (SVD) of all the data and fills the zero cells each time. When the reconstruction is essentially the same from one iteration to the next, it has converged. Additionally, the singular values are regularized so that only the most informative singular vectors impute the zero cells.

Performing statistical tests on the transformed matrix is the ultimate goal, but whether one method is truer to the original data structure is unknown. Soft imputation is unbiased and uses the structure of the data, but it has the potential to introduce arbitrary values that may reconstruct well on the whole while introducing noise and artificial structure for individual features. It also uses more parameters, which must be well tuned. Pseudocounts are simple, but also introduce artificial variance, although unbiasedly. Choosing an unsuitable zero-handling method can misrepresent the data, adding noise and inflating false positives.

Explored Question

Which method is more sensitive to a simulated effect-size? A higher sensitivity (power) at a false discovery rate (FDR) below 0.05 when testing for a group difference in the CLR-transformed data indicates superior imputation. *I hypothesize that soft imputation will better recapitulate the original multivariate structure than a pseudocount evidenced in a higher sensitivity than the pseudocount method when FDR is below 0.05.*

Simulation Setup

To test which method is better, I simulate sparse compositional data and introduce an effect. I simulate with the zeros built in instead of adding them artificially, as this is more representative of biological data. Both methods use many parameters, and the simulation step also uses many, so I use a grid search to identify the best ones for each method.

Generative Model. Counts are modeled by sampling probability vectors from a Dirichlet distribution, here called θ. θ is each sample's true composition. For each sample, the Multinomial step samples N_i times from that sample's probability vector, θ_i, this simulates the sequencing depth of realistic library sizes which varies across samples, N. This step also introduces zeros by chance and the fixed library size means counts are negatively correlated by construction (as a feature that gets more draws leaves fewer for others). The library sizes N are a simple sample from a Log-Normal with a log-scale mu (median) and sd (multiplicative).

$$X_i \sim \mathrm{Multinomial}(N_i, \theta_i)$$
$$\theta_i \sim \mathrm{Dirichlet}(\alpha)$$
$$N_i \sim \mathrm{lognormal}(\mu, \sigma)$$
$$P_{ij} = \frac{X_{ij}}{\sum_{k=1}^{p} X_{ik}}$$
$$\text{where } X \in \mathbb{N}^{n\times p}, \quad i = 1, ..., n, \quad \text{and} \quad j = 1, ..., p$$

Parameters. I simulated $n = 100$ samples (50 per group) and $p = 5000$ features. The mean composition m was drawn from a log-normal distribution (log-scale mean 0, sd 1.5) and normalized to sum to 1, and the Dirichlet parameter was $\alpha = m \cdot \mathrm{concentration}$. I could have used alpha on it's own, but concentration allowed me to control variation between samples, essentially how tightly samples cluster around m. Library sizes N_i had a median of 10^6 reads and a log-scale sd of $\frac{\ln(2)}{2}$ (a 2-fold range covers about 95% of samples); I clipped these values to $[10^5, 10^7]$. Features that were

zero in at least 90% of samples were removed (a prevalence filter), removed roughly 700 features.

Effect. For samples in group A, the probability θ_{ij} of each differential feature was multiplied by the fold change fc and each θ_i was renormalized to sum to 1; group B was unchanged. The effect is increases group A features and the renormalization shifts all other features down slightly as a result.

Zero handling. For the pseudocount, every zero in the relative abundances P was replaced by a constant c and nonzero entries were left unchanged. For soft imputation, I first converted values to the log-scale and then set zeros to half the smallest observed value of their feature. Each iteration then centered the columns, took the SVD, shrank the singular values by λ times the largest one, kept the top r components (r = rank fraction $\times \min(n, p)$), and replaced the zeros with the reconstruction. Because we assume a zero means a value was below detection, imputed values were capped to be below the feature's smallest observed value, and observed entries were never changed. Iteration stopped when the change in the sum of squared imputed log-values fell below 10^{-3} or after 100 iterations. Both methods were followed by the CLR transform.

Testing and evaluation. For each feature I ran a two-sample t-test between the groups on the CLR values, adjusted the p-values with the Benjamini-Hochberg procedure and labeled features with $q < 0.05$ as discovery. Because the differential features are known, I computed sensitivity $= \frac{\text{TP}}{\text{TP} + \text{FN}}$, specificity $= \frac{\text{TN}}{\text{TN} + \text{FP}}$, and FDR $= \frac{\text{FP}}{\text{TP} + \text{FP}}$. In the grid searches these were averaged over five simulation seeds, and the best setting of each method was the one with the highest sensitivity among settings with FDR < 0.05. The two methods were then compared at their best settings across fold changes, with one simulated dataset per fold change.

Results

TODO: Table 2 (journal-ready): sensitivity, specificity, and FDR for each method at its best setting, with an informative title and units. Use meaningful labels, not code names (e.g. "Sensitivity", not `sens`).

TODO: Figures: replace code names on the axes, colorbars, and captions (`sens`, `fdr`, `pcount`, `lam`, `rank_frac`, `value`) with readable labels (Sensitivity, FDR, pseudocount, λ, rank fraction). Captions should state what is plotted and the simulation settings (e.g. fc, seeds). Extra figures and tables go in Supplemental Materials as Figure S1, Table S1, etc.

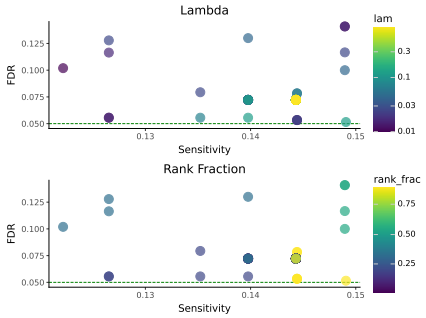


Figure 1: Soft imputation grid search over λ and rank.

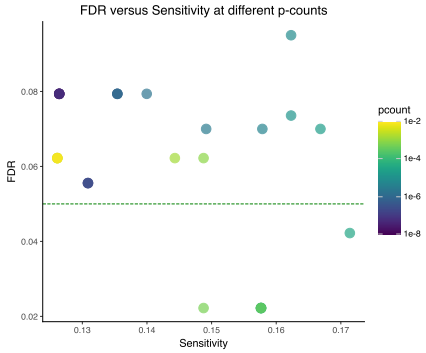


Figure 2: Pseudocount grid search.

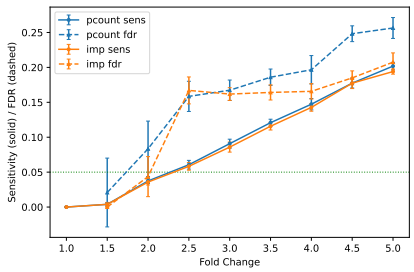


Figure 3: Pcount and soft imputation FDR versus sensitivity (power).

Discussion

TODO (10 pts): Interpretation: interpret the results in the context of the original question. Was the hypothesis supported, and why do the methods differ or not? End with a conclusion.

TODO (5 pts): Limitations of the simulation or of the question examined (single data-generating model, how the zeros arise, the range of fold changes, the effect size and sparsity values explored, and tuning on the same simulation model), and next steps.

References

References

1. Hron, K., Templ, M. & Filzmoser, P. Imputation of missing values for compositional data using classical and robust methods. *Comput. Stat. Data Anal.* **54,** 3095–3107 (2010).

Supplemental Materials

TODO: Optional extra figures and tables (Figure S1, Table S1, ...), each referenced in the text.

Appendix

Code

TODO (25 pts): Reproducibility: include the code for every simulation, analysis, figure, and table in the report, with clear comments, and exclude unused code. State the random seed(s) explicitly. The code will be run by the grader, so check that it runs without error from a clean environment and gives exactly the numbers in the report. The assignment names R or SAS but allows other code.

```python
1
```